AeroLab: An ESP32-Based Embedded Control and Sensor Telemetry Demonstrator

Manav Sharma

School of Electronics Engineering, Vellore Institute of Technology, Vellore, India

Corresponding author: Manav Sharma

Abstract— This paper presents AeroLab, an educational embedded-control demonstrator integrating an ESP32, dual ultrasonic ranging, inertial and environmental sensing, a servo actuator, a browser dashboard, and MATLAB/Python engineering scopes. A nominal second-order model is discretized at 50 ms; MATLAB computes discrete LQR and observer coefficients that are exported to firmware. A two-sided cumulative-sum monitor is evaluated in simulation using injected drift, while sensor freshness and proximity-state transitions are assessed through bounded USB telemetry observations. Following a physical connection repair, both ultrasonic channels returned fresh echoes on 40 of 40 sampled bridge polls during a 10-s clear-scene interval. In a separate target interaction, the servo-mounted channel produced caution and critical states and recovered to nominal after target removal; the fixed channel produced a close-range critical transition and recovery. Later monitoring found intermittent freshness on the servo-mounted channel, so continuous reliability is not claimed. The servo scan estimates a commanded angular sector from one range sensor; it does not estimate continuous two-dimensional hand pose. Results distinguish model simulation, synthetic demonstration, and live telemetry, and identify calibration and independent actuator feedback as necessary next steps.

Index Terms— *Embedded control, ESP32, discrete LQR, ultrasonic sensing, telemetry, CUSUM, model-based monitoring.*

I. INTRODUCTION

Low-cost embedded platforms make it possible to demonstrate sensing, feedback, alarms, and data visualization in one bench-scale system. The scientific value of such a platform depends not only on whether a dashboard updates, but also on whether each plotted signal has a clearly stated origin. Digital-twin literature describes a spectrum of physical-to-model coupling and lifecycle data integration [1], [2]. AeroLab is positioned more narrowly: it is an educational embedded control and telemetry demonstrator with a model-based software monitor, not a validated aircraft twin or flight controller.

The prototype combines an ESP32 firmware application, a MATLAB design script, two browser dashboards, a USB telemetry bridge, and desktop Python/MATLAB scopes. The firmware reads two ultrasonic channels and supporting inertial/environmental sensors, applies freshness-aware alarms, runs a normalized sampled-data controller, and emits JSON. MATLAB supplies the assumed plant model and generated gains. Simulated traces and live sensor packets share visualization software but are treated as different evidence classes.

This paper makes four practical contributions: (1) documents the end-to-end hardware/software architecture and signal interfaces; (2) specifies the discrete LQR, observer, saturation, and residual/CUSUM implementation; (3) reports bounded post-repair live-ranging observations alongside MATLAB simulation results; and (4) defines a reproducible validation path for calibration, timing, tracking, and detector performance. Unlike a paper claiming experimentally validated fault diagnosis, the present work does not have independent physical actuator-position feedback or a calibrated sensor data set [3], [4].

II. RELATED WORK

The digital-twin concept originated in aerospace lifecycle engineering and has been developed into industrial architectures linking a physical asset, model, data, and services [1], [2]. These frameworks motivate explicit synchronization and data provenance. AeroLab shares the integration goal but currently lacks a measured physical actuator state against which its model can be continuously corrected. The "twin" terminology is therefore used for the embedded model component, not as a claim of high-fidelity synchronization.

State estimation and residual-based monitoring have a long control and signal-processing history [5], [6]. CUSUM accumulates signed deviations to detect persistent shifts while tolerating small nominal fluctuations [6], [7]. The cited 2025 IEEE Access study applies a digital-twin diagnostic framework to its own monitored system [3]; it motivates the selected comparison, but its experimental performance cannot be transferred to AeroLab. Similarly, LQR theory provides an optimal state-feedback design under a specified model and quadratic cost [8]. The present paper compares design and evidence scope, not numerical performance against those published systems.

TABLE I. RELATED METHODS AND AEROLAB'S EVIDENCE SCOPE

Literature area	Established focus	AeroLab relation
Digital twins [1], [2]	Physical/model integration and lifecycle data	Bench-scale subset; no demonstrated continuous state correction
Twin-based diagnosis [3], [4]	Model residuals for monitored equipment	Software offset test; no independent actuator measurement
CUSUM [6], [7]	Sequential detection of persistent shifts	Two-sided detector implemented; evaluated here in simulation
Discrete LQR [8]	Optimal feedback for a specified model and cost	ZOH design and gain export; plant parameters remain assumed

III. PLATFORM ARCHITECTURE

Fig. 1 separates the physical acquisition path from the model/simulation path. In the physical path, the ESP32 reads sensors, updates its internal state and alarm logic, commands the servo and indicators, and emits telemetry. In the analysis path, MATLAB computes the assumed plant matrices and gains and generates model-based figures. A local Python bridge can read the ESP32 serial stream and provide the latest packet to the browser and native scope applications. All

scope panels inherit the provenance and limitations of their source packet; visualization does not validate a measurement.

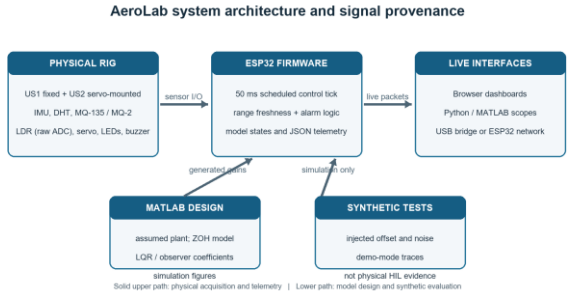


Fig. 1. AeroLab architecture. The upper path carries physical sensor input and live firmware telemetry; the lower path contains model design and synthetic software evaluation.

TABLE II. PRINCIPAL HARDWARE AND SIGNALS

Subsystem	Interface / pin	Reported quantity
Fixed ultrasonic US1	TRIG 5 / ECHO 18	Filtered range and echo health
Servo ultrasonic US2	TRIG 19 / ECHO 23	Sector-associated range
SG90 servo	PWM GPIO 13	Command only; no encoder
MPU-6050 / OLED	I²C 21 / 22	Attitude / local display
DHT11 / MQ-135 / MQ-2	GPIO 4 / ADC 32, 35	Environment; raw gas ADC
LDR / LEDs	ADC 34 / GPIO 26, 27, 14	Raw light ADC; 3 alarm colors

The ultrasonic channels use GPIOs configured for 3.3-V ESP32 logic. Common 5-V echo outputs require a correctly rated level shifter or divider; a shared reference ground is essential. Servo and gas-sensor heater supply requirements should be met independently of signal wiring [10]. The installed module markings and exact analog front-end remain hardware-specific and should be checked against their component documentation.

For a pulse-echo module, the nominal distance calculation follows d=c_sound·t_echo/2, where division by two accounts for the outgoing and return paths [11]. Firmware pulse diagnostics preserve the distinction between a timeout and an accepted echo; the former is not converted to a plausible 120-cm measurement. The configured 12-ms timeout corresponds to roughly 206 cm at 343 m/s, while the application accepts up to 200 cm to leave margin. The current acquisition cycle samples each channel nominally at 50 ms, with a 15-ms gap between channel triggers. Given two possible 12-ms echo windows, that leaves limited scheduling headroom for the rest of the loop. Actual loop duration and missed deadlines have not been instrumented and should be measured before claiming deterministic 20-Hz operation.

IV. DISCRETE CONTROL AND MODEL MONITOR

The design script assumes a second-order normalized plant with natural frequency ω_n=89 rad/s and damping ratio ζ=0.45. No identification data or fitting procedure is present in the project; these are design parameters, not empirically identified properties of the physical SG90 assembly. With x_1 as normalized displacement and x_2 as its rate, the continuous model is

$$\dot{x} = [0 \;\; 1;\; -\omega_n^2 \;\; -2\zeta\omega_n]\, x + [0;\; \omega_n^2]u, \quad y = [1 \;\; 0]x.$$

MATLAB applies exact zero-order-hold discretization at T_s=0.05 s (20-Hz nominal control schedule), producing A_d and B_d. The design uses Q=diag(100,1), R=0.01, a discrete LQR gain K, a DC prefilter $\bar{N}$, and a full-order Luenberger observer with poles derived from −120±j120 rad/s. Generated values are exported to gnc_gains.h so the firmware and analysis script share coefficients. The normalized control input is saturated to [0,1]:

$$x[k+1] = A_d\, x[k] + B_d\, u[k].$$
$$u[k] = sat_[0,1](\bar{N}r[k] - K\,\hat{x}[k]).$$

The checked-in values are K=[−0.422821, 0.000667], $\bar{N}$=0.577179, and L=[−0.186497, 8.481368]^T. The nominal discrete closed-loop poles are 0.3074 and approximately 0.0000, both inside the unit circle; this is a property of the assumed, unsaturated discrete model. The script also computes full-rank controllability and observability matrices for the continuous realization. The feedback sign convention follows the implementation u=$\bar{N}$r−K$\hat{x}$; changing it without changing the sign of K would alter the controller. A faster 1-ms propagation model is used internally between 50-ms controller updates. The servo output is computed from normalized internal state and is a PWM command, not an independently measured shaft angle.

TABLE III. DESIGN AND SIMULATION CONFIGURATION

Item	Value	Interpretation
Plant parameters	ω_n=89 rad/s; ζ=0.45	Assumed, not identified from hardware
Controller period	T_s=50 ms	20-Hz nominal schedule
LQR weights	Q=diag(100,1); R=0.01	Normalized-state design choices
Discrete poles	0.3074, ≈0	Nominal linear model stability
Observer poles	e^(−120±j120)T_s; |z|≈0.0025	Pole placement, model-based observer
Command bounds	0≤u≤1	Software saturation; not voltage

Fig. 2 is the MATLAB simulation generated from the firmware's exported discrete coefficients and includes the [0,1] command limit. It is a model simulation, not an oscilloscope trace from the servo. The source script also contains continuous-time, discrete linear, saturated-discrete, PID, and unity-feedback comparisons; these represent different designs and should not be collapsed into one empirical "best controller" claim. In particular, physical settling time, overshoot, and control accuracy require an external angle reference and repeatable input trials.

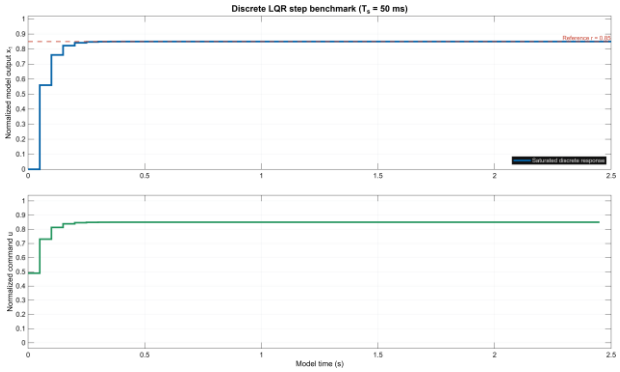


Fig. 2. MATLAB-generated discrete LQR simulation using the exported firmware coefficients. The upper trace is normalized model state; the lower trace is the normalized saturated command. No physical servo position is measured.

To provide an independent plotting path, a short Python reimplementation uses the exact rounded A_d, B_d, K, and $\bar{N}$

values in the checked-in firmware header. The recursion applies the same state-feedback sign convention and clips u to [0,1]. Its trace converges rapidly under the assumed model; the plot is a software cross-check of the coefficient export and is not a second hardware experiment.

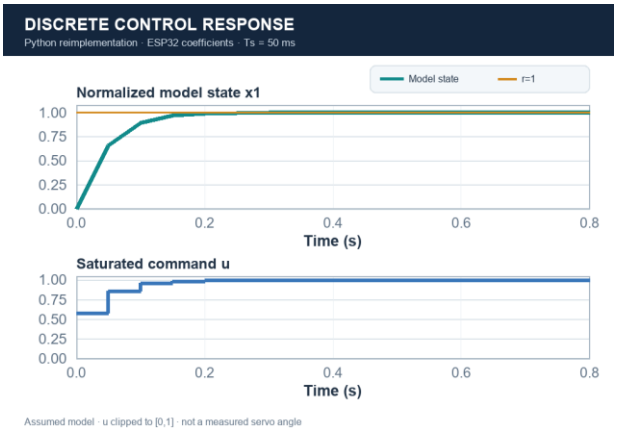


Fig. 3. Python-generated discrete response using the firmware-exported matrices and gain. The plotted state and saturated command are normalized simulation quantities; the command is not measured servo motion.

The fault-monitoring experiment defines signed discrepancy $e[k]=y_p[k]-y_t[k]$, absolute residual $\rho[k]=|e[k]|$, and two one-sided CUSUM statistics:

$$S^{+}[k]=max(0,S^{+}[k-1]+e[k]-\kappa),$$
$$S^{-}[k]=max(0,S^{-}[k-1]-e[k]-\kappa).$$

The shared settings are $\kappa=0.020$, $h=0.060$ for CUSUM, and $\rho_th=0.100$ for the direct residual threshold. These thresholds are implementation parameters in normalized model units. The MATLAB simulation injects a +0.145 offset at 2.5 s and adds Gaussian measurement noise with $\sigma=0.003$. In that scripted scenario, an abrupt offset exceeds the direct residual limit at the first faulty evaluation; this is a conditional response to a deliberately large synthetic step, not an estimate of real diagnostic latency.

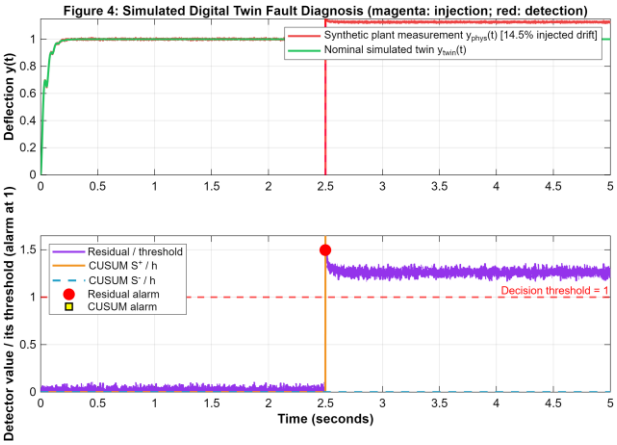


Fig. 4. MATLAB fault-injection simulation: 0.145 synthetic offset at 2.5 s; residual and CUSUM traces normalized by their thresholds. The traces are generated by software and are not physical board measurements.

A second, independently rendered Python trace illustrates one configured detector scenario using a seeded noise stream. It shows the programmed residual step crossing both normalized limits at the first post-injection sample. The reproduction supports algorithm interpretation and plot-pipeline verification; it does not estimate field detection latency or false-alarm probability.

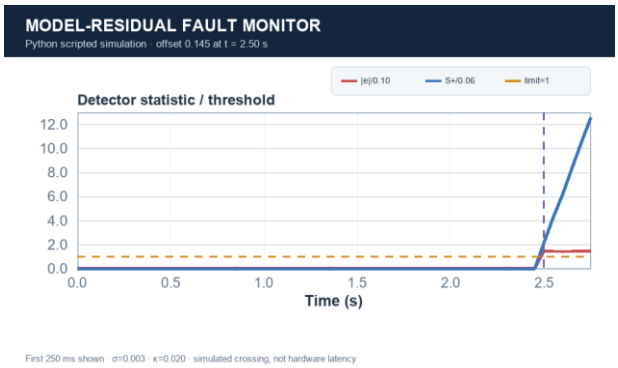


Fig. 5. Python-generated residual/CUSUM scenario with $\sigma=0.003$, $\kappa=0.020$, a 0.145 offset at 2.50 s, and configured thresholds 0.100 and 0.060. Scripted simulation only.

A key implementation boundary is that firmware x_1, $\hat{x}_1$, and y_twin are propagated from internal model states using the same command. There is no encoder or independent actuator-output sensor. Thus the firmware residual is not a measured physical tracking error. The MATLAB software experiment adds the fault term to its synthetic plant measurement; it evaluates algorithm behavior under that modeled discrepancy. A physical fault-detection claim requires an independently measured output channel and a defined fault-injection protocol [6]–[8].

V. FIRMWARE AND VISUALIZATION

The firmware provides six modes: all-sensor HUD; servo-sector scan/lock/reacquire; 180-degree radar sweep; LQR and model monitor; alternating step benchmark; and IMU attitude gimbal. The 50-ms schedule is nominal: it does not establish measured execution jitter or end-to-end telemetry latency. JSON packets carry source, mode, alarm reason, sensor-health flags, range ages and diagnostics, raw/filtered values, model states, control effort, and servo command. The USB bridge reads the serial JSON stream and serves the latest packet locally. The browser dashboards and desktop scopes display live firmware data or explicit synthetic demonstration data.

The dashboard brings together flight-mode controls, alarm status, sensor fields, a 3-D cockpit view, and linked plots. Fig. 6 presents a project dashboard render beside a separately recorded ESP32 packet summary so the interface and hardware evidence can be inspected together without implying they were captured at the same instant. The archived UI render itself shows NO LINK / AWAITING TELEMETRY; it documents the implemented layout, not a live-data result. The adjoining snapshot is the latest dated packet record and reports US1 fresh while US2 has no fresh echo.

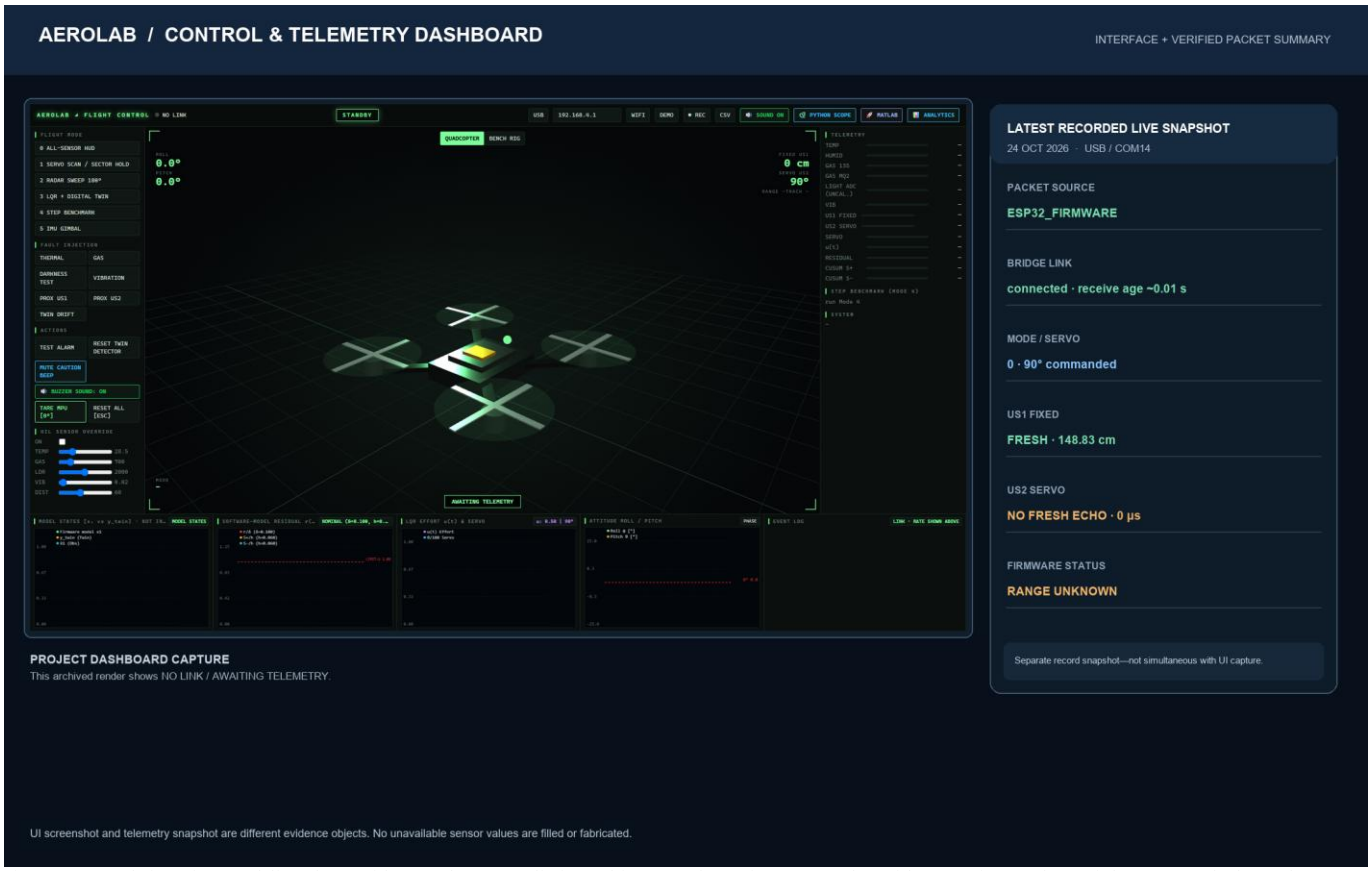


Fig. 6. AeroLab interface and live-data evidence, shown as distinct objects. Left: project UI render with no active packet. Right: separately logged COM14 snapshot (mode 0, servo command 90°, US1 fresh at 148.83 cm, US2 timeout, RANGE UNKNOWN). Firmware-reported range is not reference-calibrated.

Python and MATLAB live scopes consume the bridge stream and plot telemetry against host receive elapsed time. Their axes do not independently prove the ESP32 sampling period. Demo mode instead generates deterministic trajectories from a seeded pseudorandom process and fixed equations; demo residuals, CUSUM, servo curves, and synthetic range values are illustrative. The dashboards now preserve unavailable model fields as gaps instead of filling them with synthetic defaults, avoiding a misleading overlay when required telemetry is missing.

The explicit DEMO signal contract uses a nominal 50-ms synthetic grid, deterministic seed, common damped reference response, programmed positive and negative offsets, noise, residual, and CUSUM state. JavaScript, Python, and MATLAB test intended frame values against golden vectors. This is a cross-implementation reproducibility check, not evidence that GUI callback clocks or hardware sample clocks are synchronized. The MATLAB offline study uses a separate sampled-data simulation with 1-ms plant/twin propagation and 50-ms control/observer updates; figure labels and paper tables distinguish those experiments.

Alarm arbitration also includes temperature, gas, vibration, and model-monitor conditions. The firmware uses an adaptive temperature threshold initialized near 38 °C, suppresses MQ-135 gas alarms for the first 60 s after boot, distinguishes an elevated-gas caution from its higher critical threshold, and reports an IMU vibration caution above 0.40 g. Model drift is considered only in the LQR/model-monitor mode or when drift injection is enabled. The LDR is reported as raw ADC; there is no calibrated light-to-lux conversion or light alarm in the current firmware. Thus a dark room may lower the ADC value, but it should not be narrated as a measured illuminance without calibration.

For ultrasonic alarms, severity is computed only from fresh valid samples. Each sensor needs two consecutive newly acquired samples in its threshold band; a missing/invalid sample clears that sensor's confirmation counter. A critical proximity state has priority over caution. Both channels are combined with logical OR for alarm activation, while the message identifies which channel triggered. If either channel is stale, the interface reports RANGE UNKNOWN, and recovery to green requires both channels to be fresh and no remaining alarm condition. This fail-safe status is intentionally different from either yellow caution or red hazard.

Ultrasonic validity is treated separately from proximity. A sample is accepted in a configured 2–200-cm window; valid readings update an exponential moving average, $d_f[k]=0.65d_raw[k]+0.35d_f[k-1]$. The most recent valid sample remains fresh for 300 ms. Two distinct fresh samples are required to confirm a proximity band. Under the current code, <14 cm is critical and 14–<28 cm is caution; missing either sensor's fresh reading yields RANGE UNKNOWN rather than silently declaring the environment clear. These are firmware decision thresholds, not calibrated collision probabilities. Fig. 7 summarizes the decision path. The 200-cm upper application limit is separate from the hand-candidate limit of 55 cm. The 999-cm no-range sentinel is deliberately outside the accepted range and must never be interpreted as a real echo.

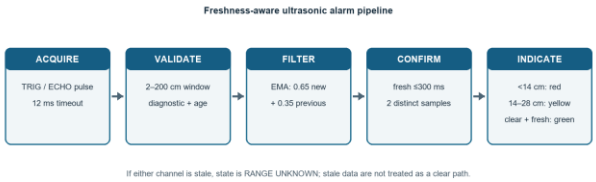


Fig. 7. Firmware range-processing path. Invalid or stale readings do not update the filter; range availability and proximity severity are separate outputs.

Mode 1 should be described as sector-level range scanning, not general hand tracking. US2 scans approximately 35°–145° in 6° commanded steps. Candidate returns must be fresh and within 2–55 cm; scan bins require coverage and the algorithm rejects broad reflections. A single ultrasonic beam and hobby servo do not measure continuous two-dimensional hand pose, classify a person, or provide a camera-like track. Mode 5 is different: it commands an IMU-based gimbal response around 90° from roll. It is not the hand-scan mode.

TABLE IV. FIRMWARE MODES AND WHAT THEY MEASURE

Mode	Behavior	Evidence / output
0	All-sensor HUD	Display and alarm monitoring
1	35°–145° sector scan / lock / reacquire	Fresh US2 range associated with commanded sector
2	Servo radar sweep	Wide angular sweep; range samples
3	LQR demand + model monitor	Normalized command and model-state residual
4	Alternating 0.15/0.85 step benchmark	Firmware model-state response metrics
5	IMU gimbal	Roll-derived servo command near 90°

VI. EVALUATION PROTOCOL AND RESULTS

Evaluation is split into (A) software design/simulation, (B) deterministic visualization checks, and (C) bounded live USB observations. A successful compile or graph self-test verifies software behavior only. Live checks use packets tagged ESP32_FIRMWARE, inspect age and echo-health fields, and count read-only bridge polls; poll counts are not necessarily the number of sensor acquisitions. The board was not flashed during the most recent post-repair paper-evidence checks.

TABLE V. SIMULATION RESULTS UNDER THE SCRIPTED SCENARIOS

Experiment	Observed result	Scope of interpretation
Residual / CUSUM; 300 runs	0% false-alarm runs; 100% detected; mean delay 0 ms for each detector	Drift=0.145; σ=0.003; injection at 2.5 s
Parameter perturbation; 200 models	100% met the script's 5% final-error criterion	ω_n ±15%; ζ ±20%; nominal controller; 60 samples

The exact printed MATLAB results are included to make the model study reproducible: all 300 Monte Carlo runs detected the programmed 0.145 offset with both the residual threshold and CUSUM, and no run alarmed before injection. The first post-injection sample crossed each threshold, so the sample-grid delay is zero. In 200 perturbed models, all trajectories met the script's finite-output, peak-output (<1.5), and final-error (<0.05) checks over 60 samples. This indicates that the selected nominal implementation tolerates those particular parameter perturbations under the scripted test; it does not quantify hardware reliability or cover unmodeled servo effects.

TABLE VI. BOUNDED LIVE ULTRASONIC OBSERVATIONS (24 OCTOBER 2026)

Test interval	Observed result	What it supports
Clear scene, 10 s	Both channels fresh on 40/40 sampled polls; green on all 40	Fresh dual-channel returns observed in this interval
US2 target interaction	Yellow at reported 20.5–25.1 cm; red at 12.0–13.3 cm; green after removal	Live caution/critical/clear transitions observed
US1 target interaction	Red at reported 6.2–9.0 cm; green after removal; yellow not sustained	Close-range transition observed; caution-band check open
Later monitor, 10 s	US1 fresh 145/145; US2 fresh 131/145; both fresh 131/145	Intermittent US2 freshness; green 91/145, caution 54/145
Later 20 s monitor	US2 fresh on 72/285 polls (25.3%); RANGE UNKNOWN on 280/285	Intermittent echo freshness in that later scene

The Python-generated evidence plot in Fig. 8 places the three documented bridge-poll intervals on one common scale. It makes the change in US2 freshness visible while retaining the sample denominators. Polling repeatedly reads the latest bridge packet; therefore these ratios describe observed packet freshness at poll time, not an independent count of ultrasonic trigger cycles.

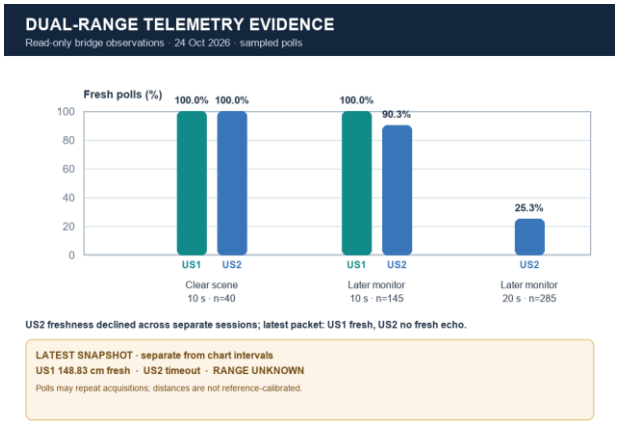


Fig. 8. Python summary of logged live bridge observations. Clear-scene and later monitor intervals are separate sessions. The latest packet snapshot is also shown, but is not part of the plotted interval denominators.

After the owner repaired a physical connection, a 10-s clear-scene interval produced fresh accepted observations from both range channels on all 40 sampled bridge polls. In a separate hands-on test, US2 produced yellow caution at reported ranges of approximately 20.5–25.1 cm, red critical at approximately 12.0–13.3 cm, then recovered to green after target removal. US1 produced red at approximately 6.2–9.0 cm and recovered to green after removal; a sustained US1 yellow response was not established. The values are firmware-reported distances and were not compared against a calibrated reference ruler/fixture.

Later observations were less stable: a 10-s monitor found US2 fresh on 131/145 bridge polls, while a separate 20-s interval found US2 fresh on 72/285 polls. During the latter interval the alarm was predominantly RANGE UNKNOWN, reflecting a missing fresh echo rather than a confirmed close object. This later result prevents a claim of uninterrupted or repeatable US2 operation. It does not identify whether the cause was target geometry, field of view, wiring, power,

interference, or the sensor module itself. A controlled clear-space and known-distance repeat is needed.

The project verification record reports successful ESP32 PlatformIO release builds, dashboard JavaScript checks, Python syntax/self-test, and MATLAB simulation/demo-contract checks. The standalone host C++ control test was not run because a compatible C++ compiler was unavailable in that environment. The project also has six saved MATLAB simulation figures. The correct interpretation is that the software path was exercised; it is not a complete validation of every hardware channel or a demonstration of aircraft control performance.

VII. DISCUSSION

Three distinctions shape the paper's interpretation. First, assumed-model simulation is useful for checking controller equations and detector code, but it cannot establish the physical SG90 dynamics. Second, raw sensor telemetry shows that a module returned a value, but accuracy and reliability require calibration and repeated trials. Third, an open scope window verifies the visualization process, but a live evidence claim also needs a fresh firmware source, valid timestamps, and explicit sensor-health indicators. These distinctions make the system easier to explain and prevent a visually persuasive interface from being mistaken for experimental validation.

The live results show why freshness metadata is valuable: the system can distinguish a clear-space echo from a missing echo and avoid interpreting stale range as a current obstacle. The observed US2 caution/critical transitions are a useful functional demonstration, while the later intermittent stream identifies an unfinished reliability question. For the same reason, a single range sensor's commanded angle should be reported as a sector estimate, not a continuous target coordinate.

The LDR channel is raw ADC and is not calibrated in lux. MQ-135/MQ-2 outputs are raw ADC counts, not gas concentration; MQ-family sensors require module-specific warm-up and calibration. The IMU depends on bias, axis convention, mounting alignment, and filtering. The servo angle is commanded. These four conditions should be stated in captions or methods whenever corresponding traces are shown. If physical tracking performance is a core claim, an independent angle/position sensor or a suitable multi-sensor/camera measurement is required.

The script's 300-run CUSUM/residual Monte Carlo study and 200-model parameter sweep are conditional simulations. Their statistics are functions of the selected random seed, noise, drift, state model, and pass/fail thresholds. They should be reported as scenario results, not empirical false-alarm probabilities or generalized robustness. The implementation should also be checked against the exact ultrasonic module's recommended acquisition period; if it is an HC-SR04 variant, its guide recommends spacing measurement cycles beyond 60 ms, whereas the firmware's nominal per-channel interval is 50 ms [11]. The physical module marking should be confirmed before changing the schedule.

VIII. REPRODUCIBILITY AND FUTURE VALIDATION

A stronger hardware study should begin with a calibration fixture and synchronized logging. For each range sensor, record raw pulse width, filtered distance, validity age, commanded servo angle, reference target distance, and alarm state at 5, 10, 20, and 50 cm. Repeat clear-scene measurements at several scan angles to expose wall reflections and mounting bias. Report absolute error, spread, missed-echo fraction, and threshold-transition repeatability. Log timestamps from one monotonic clock and measure packet rate, interarrival jitter, and stale-data duration rather than inferring them from a configured timer.

For controller validation, instrument servo output with an independent angle reference, apply repeatable steps in a safe fixture, and identify the physical response from recorded input/output data. Separate model fitting trials from held-out validation trials [12]. For fault monitoring, tune thresholds on nominal training data, inject controlled sensor/model faults, and report misses, false alarms per unit time, and detection delay with confidence intervals. For hand interaction, define target trajectory and speed and score sector localization and reacquisition; continuous pose estimation requires appropriate sensing geometry. These steps are proposed future work, not completed experiments.

For reproducibility, archive the firmware binary/hash, source revision, MATLAB release, Python dependencies, bridge configuration, raw CSV logs, sensor module markings, wiring photographs, and calibration references. Plot titles/captions should encode source type—SIMULATION, DEMO, or LIVE TELEMETRY—and carry units and validity information. A measured-performance figure should include the reference sensor and uncertainty, not only a dashboard screenshot.

IX. CONCLUSION

AeroLab integrates an ESP32 firmware controller, dual ultrasonic proximity channels, inertial and environmental sensing, a model-based monitoring path, and browser/Python/MATLAB visualization. The MATLAB-generated discrete LQR and injected-drift figures demonstrate model and software behavior under explicit assumptions. Post-repair live checks observed dual-channel fresh echoes in one clear-scene interval and yellow/red/green response from the servo-mounted channel; the fixed channel showed close-range red and clear recovery. Later US2 monitoring was intermittent, so continuous reliability and calibrated ranging remain unproven. The current Mode 1 is a sector-level ultrasonic scan rather than continuous hand-pose tracking, and the model residual lacks independent physical actuator feedback. The paper's principal result is an integrated, inspectable demonstrator with clearly bounded simulation and live evidence, together with a concrete path toward calibrated evaluation.

ACKNOWLEDGMENT

The author thanks the School of Electronics Engineering, Vellore Institute of Technology, for the academic setting in which the demonstrator was developed.

REFERENCES

[1] E. H. Glaessgen and D. S. Stargel, “The digital twin paradigm for future NASA and U.S. Air Force vehicles,” in Proc. AIAA/ASME/ASCE/AHS/ASC Structures, Structural Dynamics, and Materials Conf., Honolulu, HI, USA, 2012, Art. no. AIAA 2012-1818, doi: 10.2514/6.2012-1818.

[2] F. Tao, H. Zhang, A. Liu, and A. Y. C. Nee, “Digital twin in industry: State-of-the-art,” IEEE Trans. Ind. Informat., vol. 15, no. 4, pp. 2405–2415, Apr. 2019, doi: 10.1109/TII.2018.2873186.

[3] A. C. S. Araujo, L. M. Rabelo, P. C. M. Lamim Filho, and W. M. Caminhas, “A digital twin approach to smart monitoring and fault diagnosis,” IEEE Access, vol. 13, pp. 148384–148395, 2025, doi: 10.1109/ACCESS.2025.3601349.

[4] R. Isermann, Fault-Diagnosis Systems: An Introduction from Fault Detection to Fault Tolerance. Berlin, Germany: Springer, 2006.

[5] R. E. Kalman, “A new approach to linear filtering and prediction problems,” J. Basic Eng., vol. 82, no. 1, pp. 35–45, Mar. 1960, doi: 10.1115/1.3662552.

[6] E. S. Page, “Continuous inspection schemes,” Biometrika, vol. 41, nos. 1–2, pp. 100–115, Jun. 1954, doi: 10.1093/biomet/41.1-2.100.

[7] NIST/SEMATECH, “CUSUM control charts,” e-Handbook of Statistical Methods, §6.3.2.3. [Online]. Available: https://www.itl.nist.gov/div898/handbook/pmc/section3/pmc323.htm.

[8] B. D. O. Anderson and J. B. Moore, Optimal Control: Linear Quadratic Methods. Englewood Cliffs, NJ, USA: Prentice-Hall, 1989.

[9] K. J. Åström and R. M. Murray, Feedback Systems: An Introduction for Scientists and Engineers. Princeton, NJ, USA: Princeton Univ. Press, 2008.

[10] Espressif Systems, ESP32 Series Datasheet, Version 5.3, 2025. [Online]. Available: https://documentation.espressif.com/esp32_datasheet_en.html.

[11] ElecFreaks, “Ultrasonic ranging module HC-SR04: User guide.” [Online]. Available: https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf. Accessed: Oct. 24, 2026.

[12] L. Ljung, System Identification: Theory for the User, 2nd ed. Upper Saddle River, NJ, USA: Prentice Hall, 1999.